Instance Segmentation

Instance Segmentation
1. Model Introduction
2. Instance Segmentation: Image
    Effect Preview
3. Instance Segmentation: Video
    Effect Preview
5. Instance Segmentation: Real-time Detection (CSI Camera)
    Effect Preview
4. Instance Segmentation: Real-time Detection (USB Camera)
    Effect Preview

Use Python to demonstrate **Ultralytics**: Instance Segmentation effects on images, videos, and real-time detection.

1. Model Introduction

Instance segmentation goes a step further than object detection, involving identifying individual objects in images and segmenting them from other parts of the image.

The output of an instance segmentation model is a set of masks or contours that outline each object in the image, along with class labels and confidence scores for each object. Instance segmentation is very useful when you not only need to know where objects are in the image, but also their specific shapes.

Model download address: Instance Segmentation - Ultralytics YOLO Documentation

YAHBOOM version factory image already has this model built-in. No manual download required

2. Instance Segmentation: Image

Use yolo26n-seg to predict images in the ultralytics26 folder.

Enter the code folder:

```
cd /ultralytics/yahboom_demo
```

Run the code:

```
python3 02.segmentation_image.py
```

Effect Preview

yolo recognition output image location: /root/yolo26_data/output/

Sample code:

```
from ultralytics import YOLO

# Load a model
model = YOLO("/root/yolo26_data/models/yolo26n-seg.pt")
# model = YOLO("/ultralytics/yolo26n-seg.pt")

# Run batched inference on a list of images
results = model("/root/yolo26_data/zidane.jpg")  # return a list of Results
objects

# Process results list
for result in results:
    # boxes = result.boxes  # Boxes object for bounding box outputs
    masks = result.masks  # Masks object for segmentation masks outputs
    # keypoints = result.keypoints  # Keypoints object for pose outputs
    # probs = result.probs  # Probs object for classification outputs
    # obb = result.obb  # Oriented boxes object for OBB outputs
    result.show()  # display to screen
    result.save(filename="/root/yolo26_data/output/zidane_output.jpg")  # save
to disk
```

3. Instance Segmentation: Video

Use yolo26n-seg to predict videos in the yolo_data folder (not videos that come with ultralytics).

> Note: JetsonNano may experience freezing and restart. If this occurs, it is recommended to try smaller videos or use yolov11/yolov5 version

Enter the code folder:

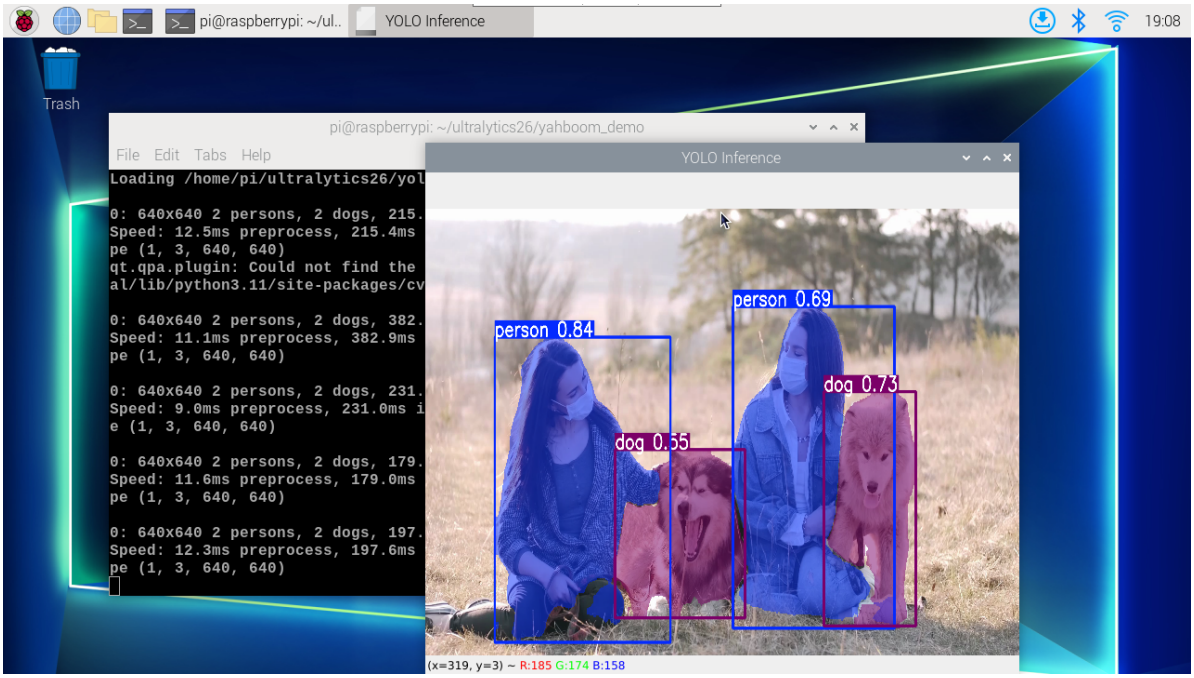
```
cd /ultralytics/yahboom_demo
```

Run the code:

```
python3 02.segmentation_video.py
```

Effect Preview

yolo recognition output video location: /home/pi/ultralytics/root/yolo26_data/output/



Sample code:

```
import cv2
from ultralytics import YOLO

# Load the YOLO model
model = YOLO("/root/yolo26_data/models/yolo26n-seg.pt")
# model = YOLO("/ultralytics/yolo26n-seg.pt")

# Open the video file
video_path = "/root/yolo26_data/people_animals_s.mp4"
cap = cv2.VideoCapture(video_path)

# Get the video frame size and frame rate
frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = int(cap.get(cv2.CAP_PROP_FPS))

# Define the codec and create a VideoWriter object to output the processed video
output_path = "/root/yolo26_data/output/02.people_animals_output.mp4"
fourcc = cv2.VideoWriter_fourcc(*'mp4v')  # You can use 'XVID' or 'mp4v'
depending on your platform
out = cv2.VideoWriter(output_path, fourcc, fps, (frame_width, frame_height))

# Loop through the video frames
while cap.isOpened():
    # Read a frame from the video
    success, frame = cap.read()

    if success:
        # Run YOLO inference on the frame
        results = model(frame)

        # Visualize the results on the frame
```

```
        annotated_frame = results[0].plot()

        # Write the annotated frame to the output video file
        out.write(annotated_frame)

        # Display the annotated frame
        cv2.imshow("YOLO Inference", cv2.resize(annotated_frame, (640, 480)))

        # Break the loop if 'q' is pressed
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break
    else:
        # Break the loop if the end of the video is reached
        break
# Release the video capture and writer objects, and close the display window
cap.release()
out.release()
cv2.destroyAllWindows()
```

5. Instance Segmentation: Real-time Detection (CSI Camera)

Use yolo26n-seg for real-time instance segmentation through CSI camera.

> Note: Please ensure CSI camera is properly connected and use docker startup script that supports CSI camera to enter container

Enter the code folder:

```
cd /ultralytics/yahboom_demo
```

Run the code:

```
python3 02.segmentation_camera_csi.py
```

> Press keyboard `q` key to exit real-time detection

Effect Preview

yolo recognition output video location: /root/yolo26_data/output/

Sample code:

```
import cv2
from ultralytics import YOLO


def gstreamer_pipeline(sensor_id=0, width=640, height=480, fps=30):
    return (
        f"nvarguscamerasrc sensor-id={sensor_id} ! "
        f"video/x-raw(memory:NVMM),width={width},height={height},framerate=
{fps}/1 ! "
        "nvvidconv ! "
        "video/x-raw,format=BGRx ! "
        "videoconvert ! "
```

```
        "video/x-raw,format=BGR ! appsink"
    )


# Load the YOLO model
model = YOLO("/root/yolo26_data/models/yolo26n-seg.pt")


# Open the camera
frame_width = 640
frame_height = 480
fps = 30
cap = cv2.VideoCapture(
    gstreamer_pipeline(width=frame_width, height=frame_height, fps=fps),
    cv2.CAP_GSTREAMER,
)

if not cap.isOpened():
    raise RuntimeError("Failed to open CSI camera")

# Define the codec and create a VideoWriter object to output the processed video
output_path = "/root/yolo26_data/output/02.segmentation_camera_csi.mp4"
fourcc = cv2.VideoWriter_fourcc(*"mp4v")
out = cv2.VideoWriter(output_path, fourcc, fps, (frame_width, frame_height))

# Loop through the video frames
while cap.isOpened():
    # Read a frame from the video
    success, frame = cap.read()

    if success:
        # Run YOLO inference on the frame
        results = model(frame)

        # Visualize the results on the frame
        annotated_frame = results[0].plot()

        # Write the annotated frame to the output video file
        out.write(annotated_frame)

        # Display the annotated frame
        cv2.imshow("YOLO Inference", cv2.resize(annotated_frame, (640, 480)))

        # Break the loop if 'q' is pressed
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break
    else:
        # Break the loop if the camera frame cannot be read
        break

# Release the video capture and writer objects, and close the display window
cap.release()
out.release()
cv2.destroyAllWindows()
```

4. Instance Segmentation: Real-time Detection (USB Camera)

Use yolo26n-seg for real-time instance segmentation through USB camera.

> Note: Please ensure USB camera is properly connected to JetsonNano

Enter the code folder:

```
cd /ultralytics/yahboom_demo
```

Run the code:

```
python3 02.segmentation_camera_usb.py
```

> Press keyboard `q` key to exit real-time detection

Effect Preview

yolo recognition output video location: /root/yolo26_data/output/

Sample code:

```
import cv2
from ultralytics import YOLO

# Load the YOLO model
model = YOLO("/root/yolo26_data/models/yolo26n-seg.pt")


# Open the cammera
cap = cv2.VideoCapture(0)
cap.set(6, cv2.VideoWriter.fourcc('M', 'J', 'P', 'G'))
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)

# Get the video frame size and frame rate
frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = int(cap.get(cv2.CAP_PROP_FPS))

# Define the codec and create a VideoWriter object to output the processed video
output_path = "/root/yolo26_data/output/02.segmentation_camera_usb.mp4"
fourcc = cv2.VideoWriter_fourcc(*'mp4v')  # You can use 'XVID' or 'mp4v'
depending on your platform
out = cv2.VideoWriter(output_path, fourcc, fps, (frame_width, frame_height))

# Loop through the video frames
while cap.isOpened():
    # Read a frame from the video
    success, frame = cap.read()

    if success:
        # Run YOLO inference on the frame
        results = model(frame)
```

```
        # Visualize the results on the frame
        annotated_frame = results[0].plot()

        # Write the annotated frame to the output video file
        out.write(annotated_frame)


        # Display the annotated frame
        cv2.imshow("YOLO Inference", cv2.resize(annotated_frame, (640, 480)))

        # Break the loop if 'q' is pressed
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break
    else:
        # Break the loop if the end of the video is reached
        break
# Release the video capture and writer objects, and close the display window
cap.release()
out.release()
cv2.destroyAllWindows()
```